A C U M E N U S

HAPI FHIR to OMOP CDM

on BigQuery

A research brief for the Chile team

ACM-FHIR-OMOP-001

Prepared April 8, 2026

Audience: informaticians, lab scientists, and data engineers

Prepared for: Chile Partner Team

 | © 2026 Acumenus, Inc.

TL;DR — the recommendation in one paragraph

Keep HAPI FHIR as the system of record for the daily Roche LIMS feed. Mirror it nightly into a **Google Cloud Healthcare API FHIR store** (or export directly to GCS as NDJSON if you prefer to keep HAPI as the only FHIR server), land the raw FHIR in BigQuery using the **Analytics V2 schema**, and then run the **FHIR → OMOP transformation as a dbt project in BigQuery**, taking the HL7/OHDSI **FHIR-to-OMOP Implementation Guide (2025 ballot)** as the canonical design spec and the **NACHC-CAD fhir-to-omop** codebase as the reference implementation to borrow mapping logic from. Orchestrate the daily run with **Cloud Composer (Airflow)**, store the OMOP Vocabulary (LOINC, SNOMED, UCUM) loaded from **Athena** in its own BigQuery dataset, and validate the CDM with **OHDSI Achilles / DataQualityDashboard**. This gives you a 100% GCP-managed pipeline, stays OHDSI-native on the transformation side, and avoids the two traps that most teams fall into: (a) choosing a FHIR-to-OMOP tool that doesn't actually target BigQuery, and (b) using a tool that outputs OMOP but isn't maintained.

1. Understanding the starting point

What you have

- **Source:** Roche LIMS (most likely cobas infinity and/or navify Lab Operations). Roche's platforms emit HL7 v2 natively; newer navify components also expose FHIR-based APIs.
- **Landing zone:** a HAPI FHIR server populated daily from the LIMS. This already does the hard work of translating Roche's representation into FHIR resources — primarily Patient, Encounter, Observation (lab results), DiagnosticReport (grouped lab reports), and Specimen.
- **Destination:** OMOP CDM in BigQuery on GCP.
- **Cadence:** daily batch is acceptable.

What OMOP needs from a LIMS feed

For a lab-dominant dataset, OMOP is a relatively narrow transformation. You are mostly populating five tables:

1. **PERSON** (demographics from Patient)
2. **VISIT_OCCURRENCE / VISIT_DETAIL** (from Encounter, or synthesized from DiagnosticReport.effective)
3. **MEASUREMENT** (the overwhelming majority of rows — every lab result Observation)
4. **SPECIMEN** (from Specimen)
5. **OBSERVATION** (for anything non-quantitative — e.g., a cancelled test, a microbiology organism identification)

Everything else in OMOP (DRUG_EXPOSURE, CONDITION_OCCURRENCE, DEATH, etc.) will likely be empty or sparsely populated unless the LIMS exports more than pure lab data. Concentrate effort on getting the MEASUREMENT transformation right, not on breadth.

2. Tooling landscape — what actually exists in April 2026

The FHIR-to-OMOP space has roughly six named tools. Most of them are either (a) maintained but don't speak BigQuery, or (b) speak BigQuery but don't output OMOP. The table below is the condensed version of that survey.

Tool	Maintained?	Outputs OMOP?	BigQuery-native?	OHDSI-native?	Verdict
HL7/OHDSI FHIR-to-OMOP IG	Yes — 2025 ballot	Design only (StructureMaps, logical models)	N/A	Yes (Vulcan WG)	Use as the spec
NACHC-CAD fhir-to-omop	Yes — commits through 2025	Yes, full	No — writes to Postgres / MSSQL	Endorsed by OHDSI IG contributors	Borrow the mapping logic
Google fhir-data-pipes	Yes — active	No — outputs SQL-on-FHIR / Parquet	Yes — DataflowRunner → BQ/Parquet	No	Use for ingestion/flattening
OHDSI ETL-German-FHIR-Core	Stale (Sept 2023)	Yes	No	Yes	Avoid — abandoned
OHDSI FhirToCdm	Minimal activity	Yes	No	Yes	Avoid — sparse docs
pyomop	Active	Yes	No	Community	Dev/test only
Cloud Healthcare API	Yes	No (FHIR store only)	Yes — native Analytics V2 streaming	No	Use for ingestion
Healthcare Data Engine + Odysseus	Active, commercial	Partial (via Odysseus)	Yes	Indirectly	Consider if managed

The key insight

No single open-source tool does "HAPI FHIR → OMOP on BigQuery" natively. Every working approach is a composition. The decision is about where you draw the line between "pre-built tool" and "SQL you own."

Three credible compositions:

- **Composition A (recommended):** GCP-managed ingestion + dbt-in-BigQuery transform. Most GCP-native. Most maintainable. Largest up-front SQL effort.
- **Composition B:** GCP-managed ingestion + NACHC tool in a VM + BQ load. Least SQL. Adds a Postgres intermediary and a Java runtime to babysit.
- **Composition C:** fhir-data-pipes + custom Parquet-to-OMOP transform. Most "cloud-native" in the Beam/Dataflow sense but you still end up writing the OMOP transformation layer.

This brief recommends Composition A; Composition B is a reasonable fallback if the team doesn't want to own SQL.

3. Recommended reference architecture (Composition A)

The diagram below summarises the recommended pipeline end-to-end. Each layer is described in the notes that follow.

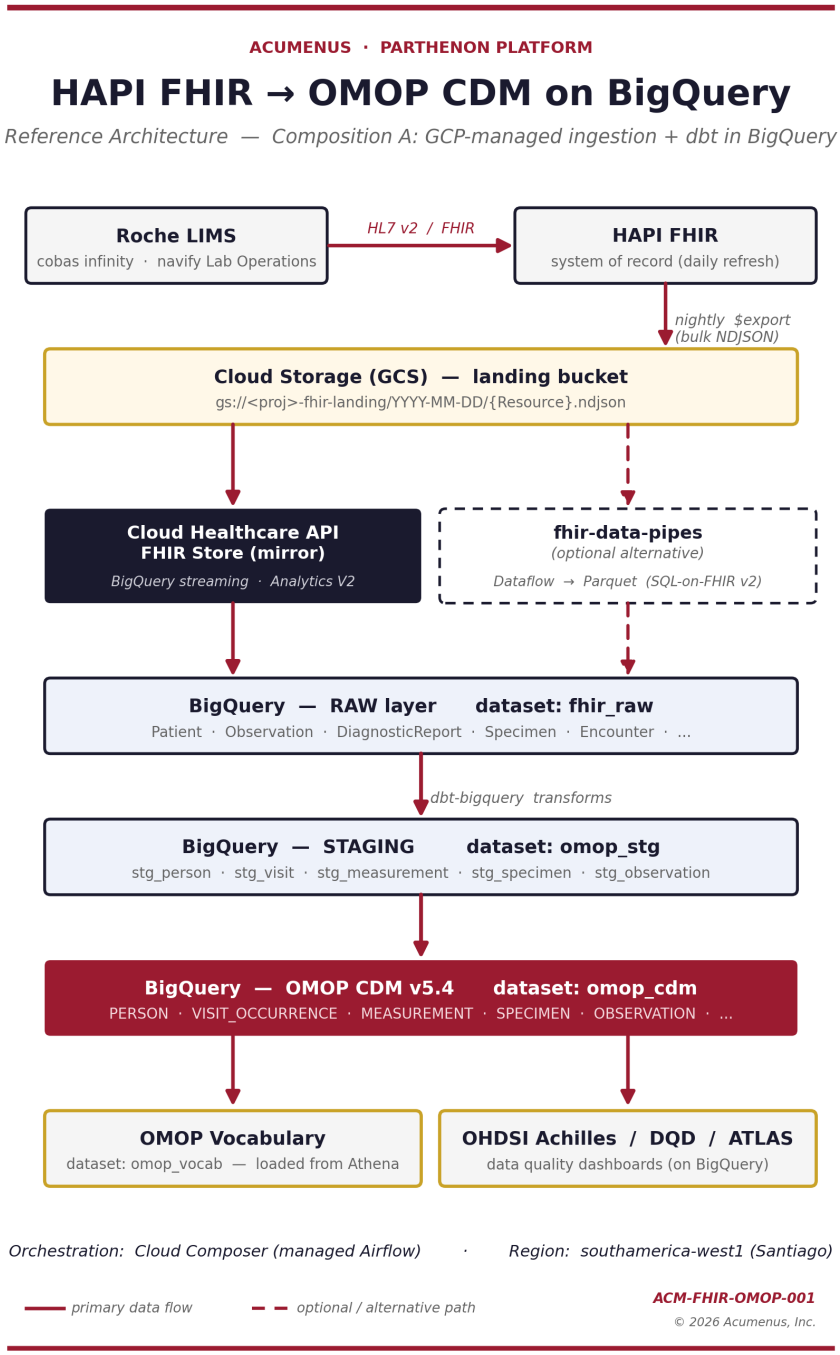

Figure 1. Reference architecture — Composition A: GCP-managed ingestion + dbt in BigQuery.

Layer-by-layer notes

Layer 1 — Ingestion from HAPI. HAPI FHIR implements FHIR Bulk Data Access ($export), which writes NDJSON files per resource type. A nightly cron-triggered POST kicks it off; resulting files can be written straight to GCS via HAPI's S3/GCS sink, or pulled by a Cloud Run job polling the $export status endpoint. NDJSON is the format everything downstream expects.

Layer 2 — Mirror to Cloud Healthcare API FHIR store (optional but recommended). The Cloud Healthcare API FHIR store supports StreamConfigs with schemaType = ANALYTICS_V2 that automatically project every resource change into BigQuery in near-real-time. If you mirror HAPI into the Cloud Healthcare API, you get a first-class, GCP-managed raw analytics layer without writing your own NDJSON flattener. The downside is running two FHIR servers. If that feels like a tax, skip this step and use fhir-data-pipes (Apache Beam) to flatten NDJSON → BigQuery instead.

Layer 3 — RAW BigQuery dataset. One table per resource type, Analytics V2 schema (normalized JSON columns with typed fields). This is the source of truth for anything downstream and should be append-only and partitioned on meta.lastUpdated.

Layer 4 — dbt transform to OMOP. This is where the real work lives. The dbt project structure should mirror the OMOP IG: stg_* models that flatten and type-cast, int_* models that resolve vocabulary joins, and omop_* models that are the final CDM tables. See Section 4 for the concrete lab-mapping logic.

Layer 5 — OMOP Vocabulary. Download the vocabulary bundle from Athena (select LOINC, SNOMED, UCUM, Gender, Race, Ethnicity, Visit at minimum), load into omop_vocab dataset. Refresh quarterly or whenever OHDSI publishes a new release.

Layer 6 — Quality checks. Run OHDSI DataQualityDashboard (DQD) against the BigQuery CDM on every nightly run — it writes a JSON report you can land in GCS and surface via Looker Studio. Run Achilles weekly for descriptive statistics. Both work against BigQuery via the DatabaseConnector R package.

Orchestration. Cloud Composer (Airflow) is the pragmatic choice because it already has operators for HTTP $export kickoff, GCS sensors, BigQuery jobs, dbt Cloud, and dbt-core. Alternatively Cloud Workflows + Cloud Scheduler is cheaper if you don't need the Airflow ergonomics.

Region. Deploy everything in southamerica-west1 (Santiago). This keeps patient data in Chile and is the most defensible posture under Chile's updated data protection law (Ley 21.719, effective December 2026). It also avoids egress costs.

4. The lab-specific mapping — the only part that really matters

Because this pipeline is LIMS-dominant, the quality of your CDM is basically the quality of the Observation → MEASUREMENT mapping. Below is the concrete field-by-field transformation, with the places most teams get it wrong.

FHIR Observation (category = laboratory) → OMOP MEASUREMENT

OMOP field	FHIR source	Notes / gotchas
measurement_id	synthesized (hash of Observation.id)	Keep a fact_relationship back to the FHIR id as

OMOP field	FHIR source	Notes / gotchas
		measurement_source_value for traceability
person_id	Observation.subject.reference → PERSON	Resolve the FHIR patient reference to your PERSON.person_id at the staging layer
measurement_concept_id	Observation.code.coding[system="loinc"].code	Join to omop_vocab.concept where vocabulary_id='LOINC' and standard_concept='S'. If the LOINC is non-standard, resolve via concept_relationship 'Maps to'. Fall back to 0 and log.
measurement_date / measurement_datetime	Observation.effectiveDateTime or effectivePeriod.start	Use effectiveDateTime for point-in-time tests. For timed collections, use effectivePeriod.start.
measurement_type_concept_id	constant	Use the “Lab result” Type Concept (historically 44818702; verify the current standard in your Athena download under vocabulary_id='Type Concept' — the exact concept has shifted between releases).
operator_concept_id	Observation.valueQuantity.comparator	Look up standard concepts for <, <=, =, >=, > in omop_vocab.concept (vocabulary_id='Meas Value Operator'). Build a small dbt seed table.
value_as_number	Observation.valueQuantity.value	Only when the value is numeric.
value_as_concept_id	Observation.valueCodeableConcept.coding	For categorical results (“Positive”/“Negative”, organism names). Resolve via SNOMED ideally.
unit_concept_id	Observation.valueQuantity.code (UCUM)	Join to omop_vocab.concept where vocabulary_id='UCUM'. If the LIMS emits non-UCUM units (common with legacy Roche feeds), build a source_to_concept_map for units.
range_low / range_high	Observation.referenceRange[0].low/high.value	Use the first reference range; keep extras in a JSON column at staging if needed.
provider_id	Observation.performer[0]	Only if a Practitioner reference is resolvable to PROVIDER.
visit_occurrence_id	Observation.encounter.reference → VISIT	Nullable — many lab results have no encounter.
measurement_source_value	Observation.code.coding[0].code	Keep the original code exactly as received.
measurement_source_concept_id	concept_id of the source code	Set to 0 if it doesn’t exist in Athena.
unit_source_value	Observation.valueQuantity.unit	Human-readable unit string (e.g., “mmol/L”).
value_source_value	Observation.valueQuantity.value or valueString	Keep the textual representation for audit.

Common traps

- **LOINC coverage is usually 60–80%, not 100%.** Plan from day one for a source_to_concept_map to cover the tail of Roche-proprietary test codes. Every unmapped test is a row with measurement_concept_id = 0.
- **UCUM units vs. Roche units.** Roche instruments often emit local unit strings ("mg/dl" lowercase, "x10^9/L" formatted oddly). Build a unit normalization lookup table early.
- **Reference ranges are sex- and age-specific.** FHIR expresses this via multiple referenceRange[] entries with appliesTo and age. OMOP MEASUREMENT has only one range_low/range_high. Either pick the applicable range at ETL time (needs person age/sex) or store the whole ranges array in a custom extension table.
- **DiagnosticReport is a grouping concept, not an OMOP table.** Don't try to force it somewhere. Accepted pattern: the DiagnosticReport becomes a synthesized VISIT_DETAIL so all child Observations share a visit_detail_id, and the report PDF/narrative goes into NOTE.
- **Specimen is its own OMOP table.** Populate it, and set MEASUREMENT.specimen_id — this is what researchers actually query on for LIMS-heavy CDMs.
- **Observation.interpretation (H, L, HH, LL, N) is not value_as_concept_id.** It belongs in MEASUREMENT.value_as_concept_id only for categorical results. For numeric labs, interpretation is usually discarded or kept in a custom extension column.

Vocabulary setup

Download from Athena with at least these vocabularies selected: SNOMED, LOINC, UCUM, Gender, Race, Ethnicity, Visit, Visit Type, Domain, Concept Class, Vocabulary, Relationship, ICD10CM (for any diagnosis data), and CDM v5.4. Load as CSV → BigQuery using bq load or the Python client. The full set is ~3 GB and fits comfortably in BigQuery at negligible cost.

5. Why not just use one of the tools directly?

A brief honest assessment of the tempting shortcuts:

"Just use NACHC fhir-to-omop." It works, and if your team doesn't want to write SQL this is the path of least resistance. But: it targets PostgreSQL/MSSQL and you'd have to stand up Cloud SQL, run the Java tool inside a Cloud Run job or VM, and then re-ship data into BigQuery. That's three extra moving parts vs. a BigQuery-native dbt project. Its mapping logic is, however, the best open-source reference and you should absolutely read it to inform your dbt code.

"Just use Google fhir-data-pipes." It's well-maintained and GCP-native, but it outputs SQL-on-FHIR v2 / Parquet views — not OMOP. You'd still need to write the OMOP transformation layer yourself. Great for staging flattened FHIR in GCS; not a FHIR-to-OMOP tool.

"Just use Cloud Healthcare API's BigQuery streaming." Same issue — Analytics V2 is flattened FHIR, not OMOP. It's the best option for getting raw FHIR into BigQuery continuously (hence Layer 2 of the architecture), but it stops there.

"Just use ETL-German-FHIR-Core." Abandoned (last commit 2023) and specific to the German Medical Informatics Initiative profile. Avoid.

"Just use the HL7 FHIR-to-OMOP IG StructureMaps directly." The IG publishes machine-readable FHIR Mapping Language StructureMaps. In theory you run them through an FML engine (matchbox, HAPI's StructureMap execution). In practice the tooling is immature, bulk performance is poor, and debugging is hard. Use the IG as a design spec, not an execution engine.

6. Build plan and rough effort

Assuming a small team (1 data engineer, 1 informatician, 0.5 clinical lead):

Phase	Duration	Deliverable
0 — Foundation	1 week	GCP project, VPC-SC perimeter, BigQuery datasets, Athena vocabulary loaded, region set to southamerica-west1
1 — Ingestion	2 weeks	Nightly HAPI $export → GCS → Cloud Healthcare API mirror → BigQuery raw layer
2 — Vocab study	1 week	Inventory of distinct LIMS test codes and units; coverage report against LOINC/UCUM; source_to_concept_map populated
3 — Core ETL (dbt)	4–6 weeks	stg_*, int_*, and omop_* models for PERSON, VISIT_OCCURRENCE, MEASUREMENT, SPECIMEN, OBSERVATION, CARE_SITE, LOCATION, CDM_SOURCE
4 — Orchestration	1 week	Airflow DAG on Cloud Composer running the full chain nightly, with alerting
5 — Data quality	2 weeks	DataQualityDashboard integration; Achilles baseline; Looker Studio board for clinicians
6 — User acceptance	2 weeks	Clinical review of a sample patient round-trip (LIMS report → FHIR → OMOP)

Total: ~13–15 weeks to a production-grade v1. The heaviest single workstream is Phase 3 (the dbt models), and within that the heaviest single model is omop_measurement. Plan accordingly.

7. Cost sanity check (order of magnitude)

For a hospital-scale LIMS (say 500k lab results per day = ~180M rows/year):

- **BigQuery storage:** ~30 GB/year for MEASUREMENT → under $1/month.
- **BigQuery query:** dbt nightly runs scanning partitioned raw FHIR → a few dollars/month on-demand, negligible on a small slot reservation.
- **Cloud Composer:** ~$350/month for a small environment. Largest line item; consider Cloud Workflows if budget is tight.
- **Cloud Healthcare API FHIR store:** storage + streaming fees, ~$50–150/month at this scale.
- **GCS landing bucket:** negligible.
- **Dataflow (if using fhir-data-pipes):** tens of dollars per day at this volume.

Total order of magnitude: $500–800/month for the full pipeline. The marginal cost of keeping the CDM refreshed daily is essentially the Composer line.

8. Governance quick notes

- **Chile's Ley 21.719** (data protection, effective December 1, 2026) aligns the country with GDPR-style obligations. Keep processing in southamerica-west1 and document the legal basis for each use of the OMOP CDM.
- **No PHI should leave the CDM layer.** If researchers need access, give them a de-identified derivative with dates shifted per patient and the NOTE table redacted. BigQuery row- and column-level ACLs make this straightforward.
- **Audit logging.** Enable BigQuery audit logs and Cloud Healthcare API audit logs from day one. Ship both to a dedicated Cloud Logging sink owned by the security team.
- **OMOP is pseudonymous by design** — there are no direct identifiers in the core tables — but PERSON.person_source_value typically holds an MRN. Hash it with a project-wide pepper before it enters BigQuery.

9. Working checklist — what to do, in order

This is the section to work through. Check items off as you go. Boxes are grouped by phase; aim to finish each phase before starting the next, with the exception of Phase 0 (Foundation) items which can run in parallel.

Phase 0 — Foundation (target: week 1)

- ☐ Create a new GCP project owned by the Chile team; name it e.g. chile-omop-prod.
- ☐ Set the default region to southamerica-west1 (Santiago) on the project.
- ☐ Enable APIs: BigQuery, Cloud Storage, Cloud Healthcare API, Cloud Composer, Cloud Run, Cloud Logging, Cloud Monitoring.
- ☐ Create a VPC-SC perimeter that contains the project; allow only the egress rules you actually need.
- ☐ Create BigQuery datasets in southamerica-west1: fhir_raw, omop_stg, omop_cdm, omop_vocab, dqd_reports.
- ☐ Create a GCS landing bucket in southamerica-west1: gs://chile-omop-prod-fhir-landing.
- ☐ Set up IAM groups: omop-admins, omop-etl-sa (service account), omop-readers, omop-researchers.
- ☐ Enable BigQuery and Cloud Healthcare API audit logs and route them to a dedicated log sink.
- ☐ Create a shared password manager / secret store entry for the HAPI FHIR service account credentials.

Phase 1 — Understand the source (target: week 1, parallel with Phase 0)

- ☐ Run a count query against HAPI FHIR to enumerate distinct Observation.code values (distinct LIMS test codes).
- ☐ Document how many of those codes are already valid LOINC codes vs. local Roche codes.
- ☐ Run a distinct query on Observation.valueQuantity.code (unit strings) and compare to the UCUM vocabulary.
- ☐ Count distinct Specimen.type values and compare to SNOMED specimen concepts.
- ☐ Confirm whether Encounter resources exist or whether you'll need to synthesize visits from DiagnosticReport.
- ☐ Pull a 7-day NDJSON sample via HAPI $export and save it locally for development and hand-mapping exercises.
- ☐ Hand-map 10 representative Observations end-to-end to OMOP MEASUREMENT rows in a spreadsheet before writing any code.

Phase 2 — Vocabulary (target: week 2)

- ☐ Register for an OHDSI Athena account if the team doesn't already have one.

- ☐ Download the OMOP Vocabulary bundle including at minimum: SNOMED, LOINC, UCUM, Gender, Race, Ethnicity, Visit, Visit Type, Domain, Concept Class, Vocabulary, Relationship, ICD10CM.
- ☐ Load the vocabulary CSVs into BigQuery dataset omop_vocab using bq load or a Python loader.
- ☐ Verify CONCEPT row counts match the Athena download manifest.
- ☐ Produce a coverage report: what percent of distinct LIMS test codes resolve to a standard LOINC concept?
- ☐ Produce the same coverage report for units against UCUM, and for specimen types against SNOMED.
- ☐ Seed an initial source_to_concept_map for the untranslatable tail of test codes (even if it's a stub that defaults to 0).

Phase 3 — Ingestion (target: weeks 3–4)

- ☐ Decide between Composition A (Cloud Healthcare API mirror) and the simpler GCS-only path. If unsure, choose A.
- ☐ Configure a nightly HAPI $export job: either a HAPI-side cron or a Cloud Run job that calls the $export endpoint and polls for completion.
- ☐ Wire $export output to write NDJSON directly into gs://chile-omop-prod-fhir-landing/YYYY-MM-DD/.
- ☐ (Composition A) Create a Cloud Healthcare API dataset and FHIR store in southamerica-west1.
- ☐ (Composition A) Configure a StreamConfig on the FHIR store with schemaType = ANALYTICS_V2 targeting dataset fhir_raw.
- ☐ (Composition A) Trigger a batch FHIR import from the GCS landing bucket into the Cloud Healthcare API FHIR store.
- ☐ Verify that raw FHIR resources appear as tables in dataset fhir_raw with the expected Analytics V2 schema.
- ☐ Add a monitoring alert on the import job (failure, lag > 6 hours).

Phase 4 — dbt transformation (target: weeks 5–10)

- ☐ Create a new dbt project (dbt-bigquery adapter) and commit it to Git.
- ☐ Set up a dbt profile that authenticates to BigQuery via the omop-etl-sa service account.
- ☐ Clone NACHC-CAD/fhir-to-omop from GitHub and read the MeasurementMapper class to capture the edge-case logic.
- ☐ Bookmark the HL7 FHIR-to-OMOP IG (2025 ballot) for reference on logical models.
- ☐ Write stg_person, stg_visit_occurrence, stg_specimen, stg_measurement, stg_observation dbt models that flatten and type-cast the raw FHIR.
- ☐ Write int_* models that resolve LOINC/UCUM/SNOMED concept IDs via joins to omop_vocab.

- ☐ Write the final omop_measurement model. Treat this as the highest-risk model and pair-review it.
- ☐ Write the remaining OMOP target models: omop_person, omop_visit_occurrence, omop_visit_detail, omop_specimen, omop_observation, omop_care_site, omop_location, omop_cdm_source.
- ☐ Add dbt tests: not_null on primary keys, relationships between MEASUREMENT.person_id and PERSON.person_id, accepted_values for type concepts, row counts per day.
- ☐ Run a full end-to-end build on the 7-day sample from Phase 1 and reconcile row counts against expectations.

Phase 5 — Orchestration (target: week 11)

- ☐ Spin up a small Cloud Composer environment in southamerica-west1.
- ☐ Author an Airflow DAG: trigger_hapi_export → wait_for_gcs → run_dbt → run_dqd → notify.
- ☐ Schedule the DAG at 02:00 Santiago time (or whatever is off-peak for HAPI).
- ☐ Add Slack / email failure notifications to the DAG.
- ☐ Run the DAG manually three times to confirm idempotency and stability.

Phase 6 — Data quality and clinical sign-off (target: weeks 12–14)

- ☐ Set up OHDSI DataQualityDashboard against the BigQuery CDM (DatabaseConnector + JDBC driver).
- ☐ Run DQD as part of the nightly DAG and write the JSON report to GCS (dqd_reports dataset or a bucket).
- ☐ Build a Looker Studio dashboard that surfaces DQD pass/fail counts and trending.
- ☐ Run OHDSI Achilles once to establish a descriptive baseline of the CDM.
- ☐ Pick 5 representative patients; have a clinical team member trace them from original LIMS report → HAPI FHIR → OMOP MEASUREMENT rows and sign off.
- ☐ Document any remaining known mapping gaps in a CDM_SOURCE.cdm_source_issues extension or a README.

Phase 7 — Go-live and handoff (target: week 15)

- ☐ Promote the dbt project to a prod target; lock down the service account permissions to least privilege.
- ☐ Add a CDM refresh SLO (e.g., “current day available by 06:00 Santiago time”) and document it.
- ☐ Hand off runbooks: how to reprocess a day, how to add a new source code to source_to_concept_map, how to bump the Athena vocabulary.
- ☐ Train at least two people on the end-to-end pipeline (bus factor ≥ 2).
- ☐ Schedule a quarterly review of LOINC/UCUM coverage and update source_to_concept_map as needed.

Ongoing (after go-live)

- ☐ Refresh the Athena vocabulary quarterly and re-run a DQD diff.
- ☐ Review unmapped test codes monthly; every new unmapped code is a candidate to add to source_to_concept_map.
- ☐ Monitor Cloud Composer and BigQuery costs; right-size the Composer environment once stable.
- ☐ Keep the HL7 FHIR-to-OMOP IG and NACHC repo on a watchlist; update the transformation when the IG goes past ballot to normative.
- ☐ Revisit governance in Q4 2026 when Chile's Ley 21.719 takes effect — confirm DPIA, data subject rights workflows, and audit posture.

Sources

Tooling

- HL7/OHDSI FHIR-to-OMOP Implementation Guide (2025 ballot)
- NACHC-CAD fhir-to-omop (reference implementation)
- Google fhir-data-pipes (SQL-on-FHIR pipelines)
- Google Open Health Stack — FHIR Data Pipes docs
- OHDSI ETL-German-FHIR-Core
- OHDSI FhirToCdm
- OHDSI WhiteRabbit / Rabbit-in-a-Hat
- pyomop

GCP ingestion and BigQuery

- Cloud Healthcare API — Streaming FHIR resource changes to BigQuery
- Cloud Healthcare API — Batch export FHIR to BigQuery
- Cloud Healthcare API — Import/export FHIR via Cloud Storage
- HL7 FHIR Bulk Data Access IG ($export)
- HAPI FHIR project site

OMOP vocabulary and lab mapping

- OHDSI Athena (vocabulary download)
- The Book of OHDSI — Standardized Vocabularies
- US Core DiagnosticReport Profile for Laboratory Results
- SNOMED/LOINC Implementation Guide — FHIR and Laboratory Data
- OHDSI Forums — measurement.value_as_concept_id mapping

SQL-on-FHIR

- SQL on FHIR v2 IG (current build)
- SQL on FHIR — npj Digital Medicine 2025

Roche LIMS interfaces

- Roche navify Lab Operations
- Roche cobas infinity laboratory solution

OHDSI tools for CDM validation

- OHDSI DataQualityDashboard
- OHDSI Achilles

Chile regulatory context

- Chile Ley 21.719 (effective Dec 1, 2026)
- Google Cloud southamerica-west1 (Santiago)

© 2026 Acumenus, Inc.